

CS 3510 Algorithms: NP-Complete: Graph and Constraint Problems

Aaron Hillegass
Georgia Tech

Decision Problems Sound Lame

Decision Problems return True or False.

Most fun problems return solutions – the result of some search.

But decision problems are easy to work with.

Are we messing things up by sticking to Decision Problems?

Decision Problems and Search Problems

Decision SAT: Given a CNF boolean expression Φ , is there a satisfying assignment?

Search SAT: Given a CNF boolean expression Φ , what is a satisfying assignment?

Claim: Search SAT is in P if and only if Decision SAT is in P .

Proof: Easy direction is assume Search SAT is in P . To solve Decision SAT, get a solution using Search SAT.

Hard direction: Assume Decision SAT is in P . How can we get a solution to Search SAT?

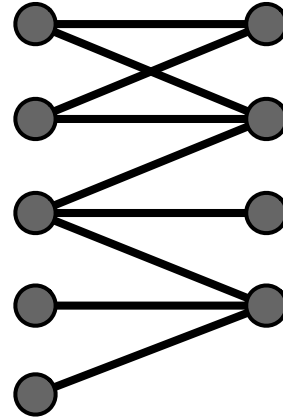
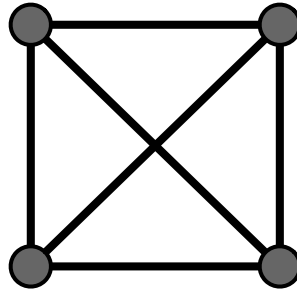
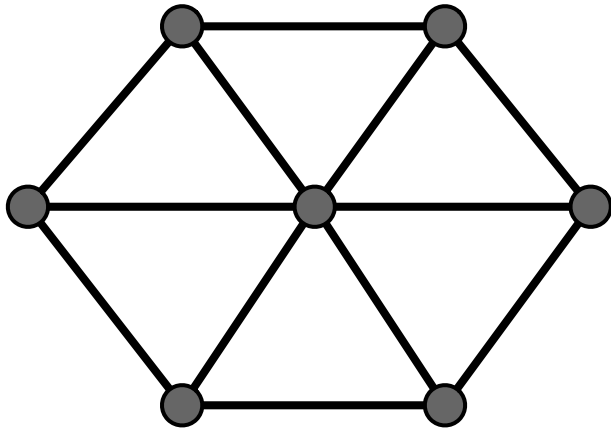
Using Decision SAT to solve Search SAT

Assuming Decision SAT is in P .

- Given an expression Φ , first use Decision SAT to see if it is satisfiable. No? You're done.
- Otherwise, find x_1 :
 - Replace x_1 in Φ with True.
 - Use Decision SAT to see if that is satisfiable?
 - Yes? $x_1 := \text{True}$. No? $x_1 := \text{False}$.
- Substitute that value in for x_1 , and repeat for x_2 . And x_3 . etc.

Can solve Search SAT by running Decision SAT polynomial times.

Given an undirected graph $G = (V, E)$ and k , is there a subset $S \subset V$ of size k such that no two vertices in S share an edge?



How do we prove INDSET is NP-Complete?

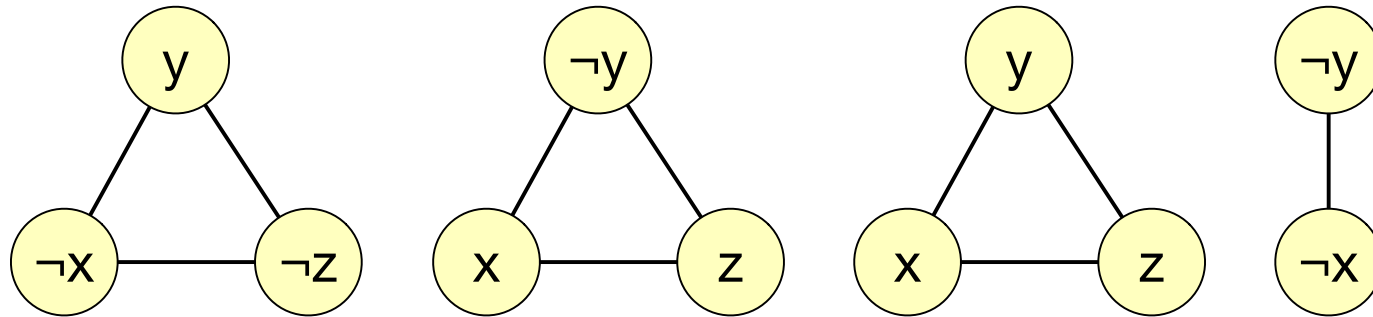
Given an undirected graph $G = (V, E)$ and k , is there a subset $S \subset V$ of size k such that no two vertices in S share an edge?

Witness?

Can we reduce 3-SAT to INDSET?

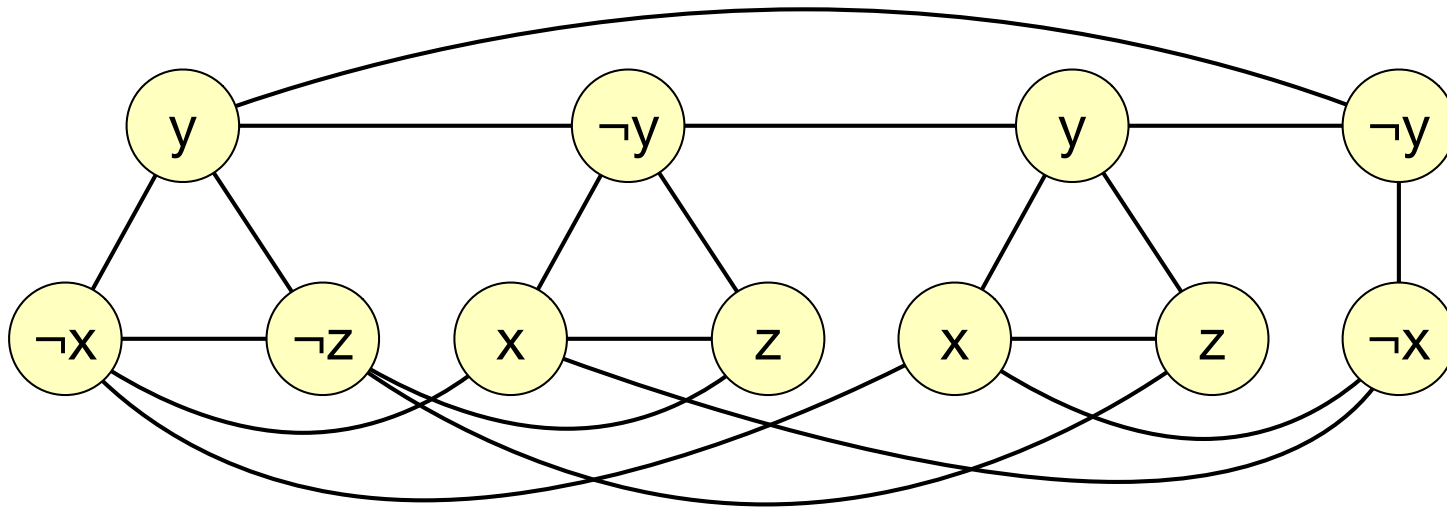
Given a boolean expression Φ , can we construct a graph G such that finding an assignment for Φ is equivalent to finding an independent set of some size k in G ?

$$\Phi = (\neg x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee y \vee z) \wedge (\neg x \vee \neg y)$$



Construction (continued)

$$\Phi = (\neg x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee y \vee z) \wedge (\neg x \vee \neg y)$$



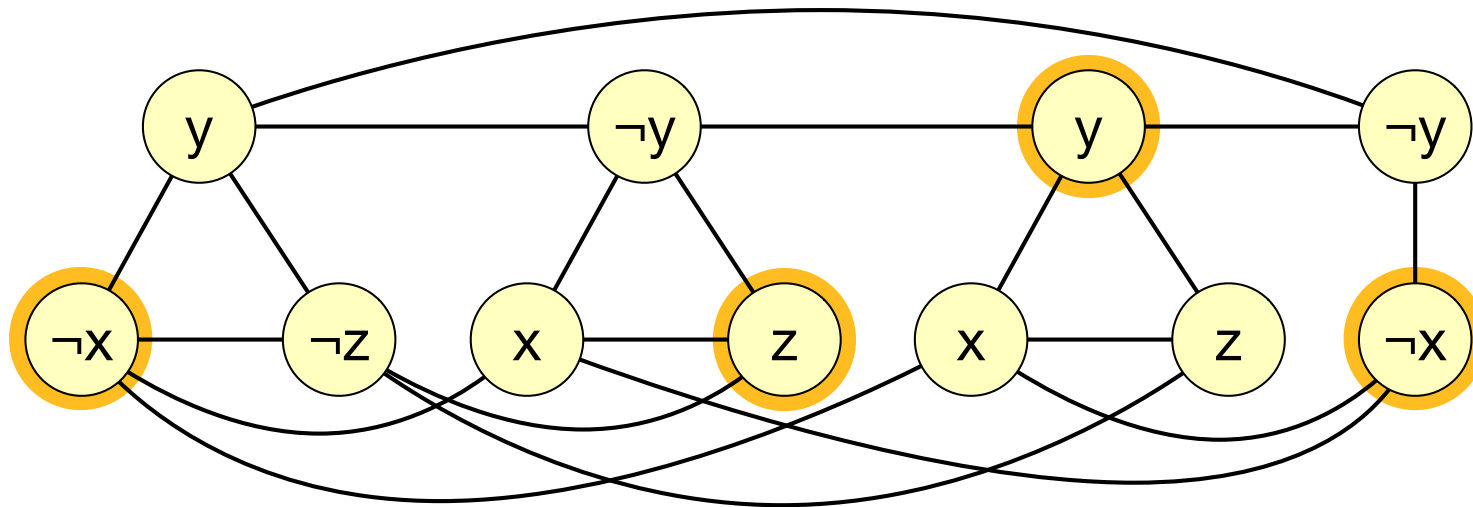
For this, how big could an independent set be?

Construction (continued)

$$\Phi = (\neg x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee y \vee z) \wedge (\neg x \vee \neg y)$$

$$x = \text{False}, y = \text{True}, z = \text{True}$$

Four clauses? Can we find an independent set of size 4 in G ?

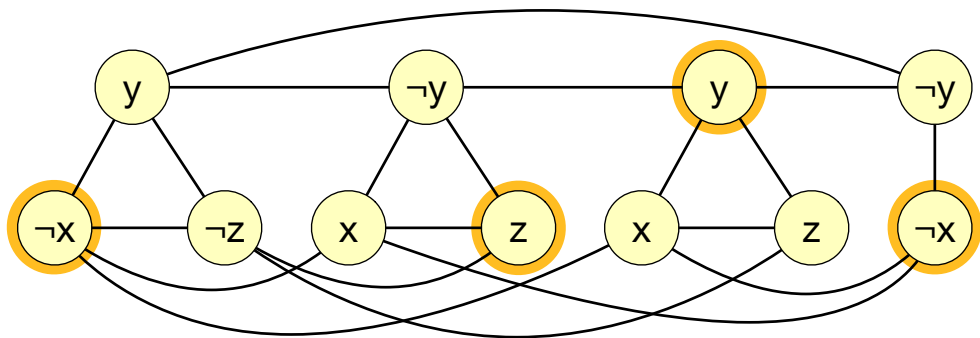


Φ , a 3CNF expression is satisfiable if and only if G has an independent set of size k .

Assume there is an assignment $a_1, a_2, \dots, a_n$ that satisfies Φ . Prove there is an independent set of size k in G .

By construction, G has at most one vertex per clause.

For each clause, the assignment must make at least one literal true, so choose one vertex corresponding to a true literal. That is an independent set of size k .

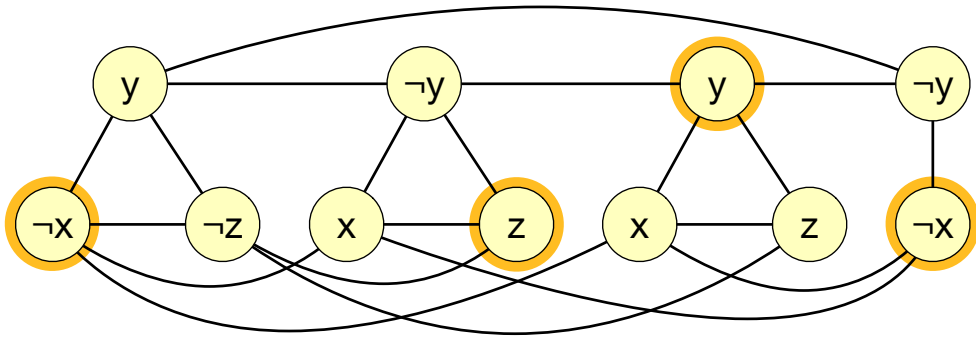


How to prove converse?

Assume G has an independent set of size k :

No two members of the set are in the same clause-triangle, so exactly one vertex per clause is chosen.

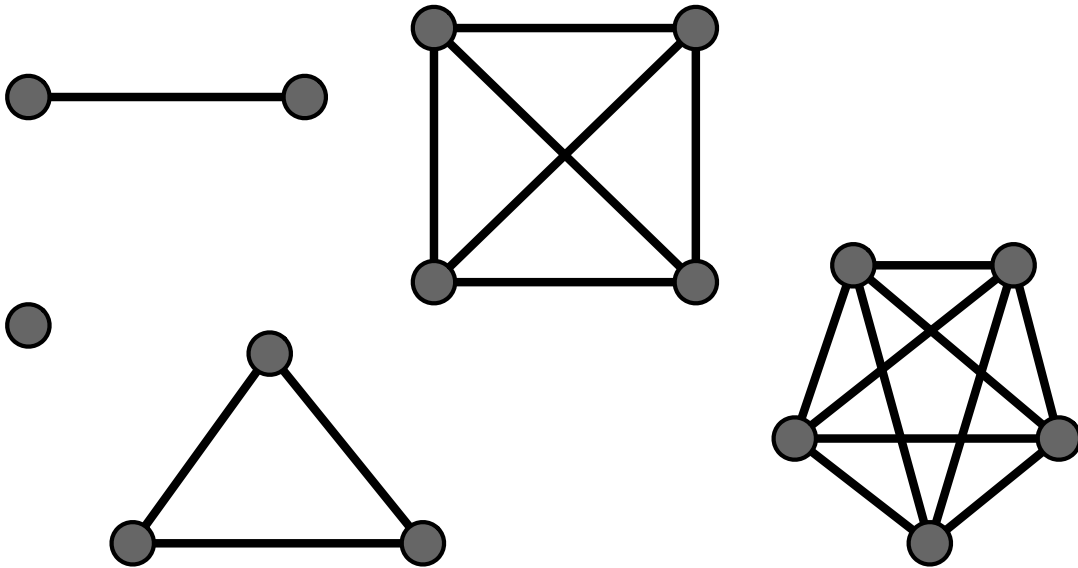
If a vertex representing $\neg x_1$ is chosen, then no x_1 vertex is chosen.



INDSET is NP-Complete.

What's a clique?

Given an undirected graph G , a clique Q is a subset of vertices where for any $u, v \in Q$, there is an edge between u and v .



What is the trivial clique?

What's CLIQUE?

Given an undirected graph G and k , is there a clique of size k ?

Verifying?

```
1 define verify( $G, k, Q$ ):  
2   if  $|Q| < k$ :  
3     return False  
4   for  $u$  in  $Q$ :  
5     for  $v$  in  $Q - \{u\}$ :  
6       if  $(u, v) \notin G.E$ :  
7         return False  
8   return True
```

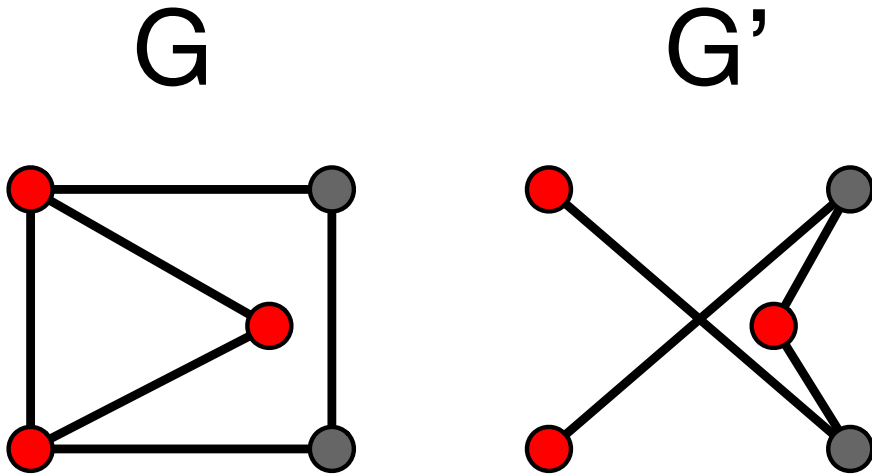
How do we prove CLIQUE is NP-Complete?

(We can use anything we already know is NP-Complete.)

INDSET reduces to CLIQUE?

Given an independent set of size k on G , can you convert it to a CLIQUE problem?

If Q is a clique in G , what is that set of vertices in the complement of G ?



Proof: INDSET reduces to CLIQUE?

Given an independent set I of G of size k , I is a clique in G' .

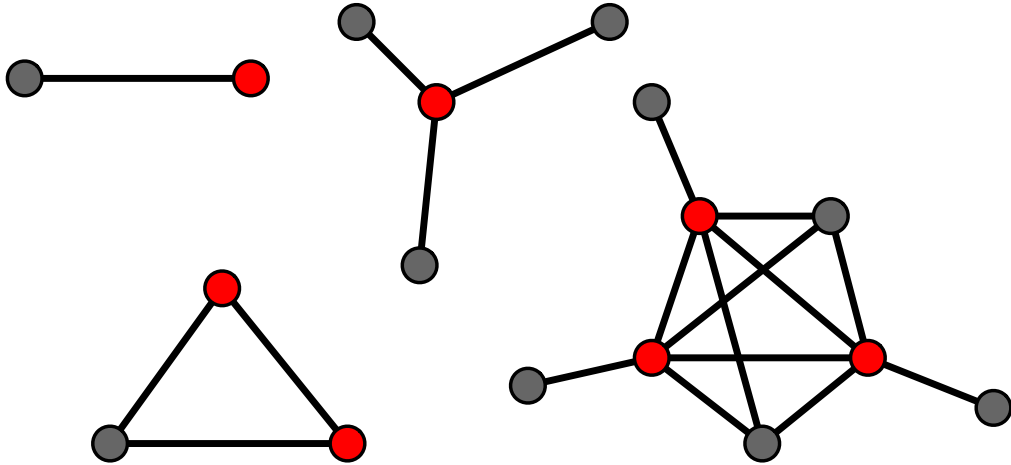
(If there is no edge between u, v in G , then there is an edge between them in G' .)

Given a clique Q in G , Q is an independent set in G' .

(If there is an edge between u, v in G , then there is no edge between them in G' .)

VERTEXCOVER

Given a graph G , is there a set of k vertices C such that every edge touches a member of C ?



What is the trivial vertex cover?

Verifier?

What do we reduce to VERTEXCOVER to
show it is NP-Complete?

3-SAT, INDSET, CLIQUE?

INDSET reduces to VERTEXCOVER

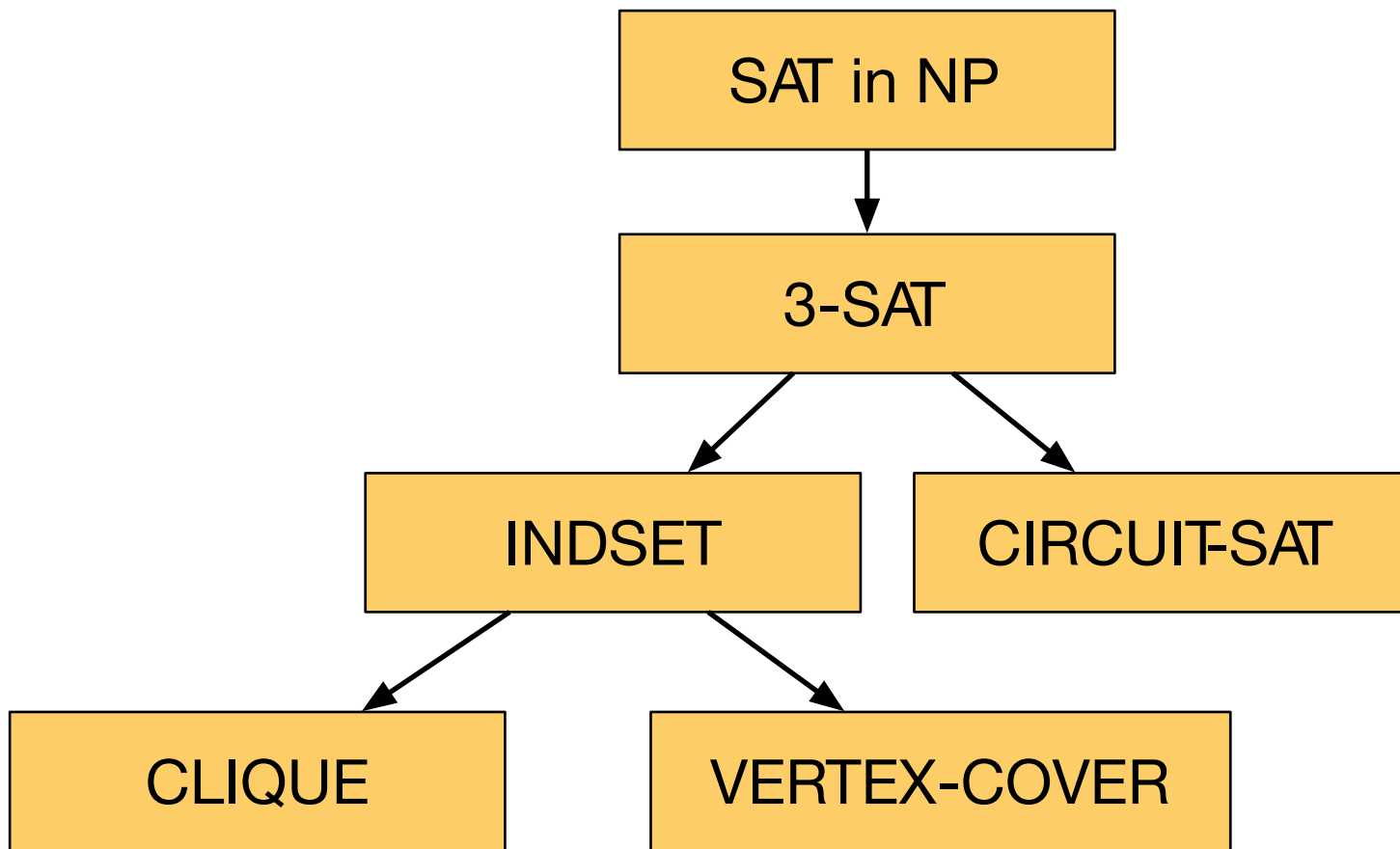
Given a graph G , there is an independent set of vertices I if and only if $V - I$ is a vertex cover of G .

Proof: If you have an edge (u, v) in G , at least one end is not in I . Thus, at least one end is in the complement of I . Thus the complement of I is a vertex cover of G . Aaron

Conversely, if C is a vertex cover, every edge (u, v) has at least one end in C , thus at most one end must be in the complement of C . Therefore the complement of C is an independent set.

How did I get here!?

22/32



Given a set S of integers and a target T , does there exist a $S' \subset S$ such that the sum of the elements in S' is exactly equal to T ?

Example: Let $S = \{8, 6, 4, 4, 2\}$, $T = 11$

Witness and Verifier?

NP-Complete? Let's work from 3-SAT.

Reducing 3-SAT to SUBSET-SUM

Given a 3CNF expression, how can we encode it as a single SUBSET-SUM problem?

Given variables x_1, x_2, x_3 . How to express the constraint “We must choose exactly one from each pair: $(x_1, \neg x_1), (x_2, \neg x_2), (x_3, \neg x_3)$ ”?

x_1	1	0	0
$\neg x_1$	1	0	0
x_2	0	1	0
$\neg x_2$	0	1	0
x_3	0	0	1
$\neg x_3$	0	0	1
Sum	1	1	1

No column can exceed 2, so interpret as base-2:

$$S = \{4, 4, 2, 2, 1, 1\}, T = 7$$

3-SAT Clauses?

Expression: $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$

x_1	1	0	0	1	0	0
$\neg x_1$	1	0	0	0	1	1
x_2	0	1	0	1	1	0
$\neg x_2$	0	1	0	0	0	1
x_3	0	0	1	1	0	1
$\neg x_3$	0	0	1	0	1	0
Sum	1	1	1	> 0	> 0	> 0

But it has to be exact.

And one sum...

Slack variables

Expression: $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$

x_1	1	0	0	1	0	0
$\neg x_1$	1	0	0	0	1	1
x_2	0	1	0	1	1	0
$\neg x_2$	0	1	0	0	0	1
x_3	0	0	1	1	0	1
$\neg x_3$	0	0	1	0	1	0
s_1	0	0	0	1	0	0
s_2	0	0	0	1	0	0
s_3	0	0	0	0	1	0
s_4	0	0	0	0	1	0
s_5	0	0	0	0	0	1
s_6	0	0	0	0	0	1
Sum	1	1	1	3	3	3

Slack variables can only contribute 2 to each sum.

Real variables must contribute one to each sum.

Will binary encoding still work?

Use base-5.

Making Sums

Expression: $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$

x_1	1	0	0	1	0	0
$\neg x_1$	1	0	0	0	1	1
x_2	0	1	0	1	1	0
$\neg x_2$	0	1	0	0	0	1
x_3	0	0	1	1	0	1
$\neg x_3$	0	0	1	0	1	0
s_1	0	0	0	1	0	0
s_2	0	0	0	1	0	0
s_3	0	0	0	0	1	0
s_4	0	0	0	0	1	0
s_5	0	0	0	0	0	1
s_6	0	0	0	0	0	1
Sum	1	1	1	3	3	3

111333 in base-5 is $1 \times 5^5 + 1 \times 5^4 + 1 \times 5^3 + 3 \times 5^2 + 3 \times 5^1 + 3 \times 5^0 = 3,125 + 625 + 125 + 75 + 15 + 3 = 3,968$

100100 = $3,125 + 25 = 3,150$

100011 = $3,125 + 5 + 1 = 3,131$

010110 = $625 + 25 + 5 = 655$

...

$S = \{3150, 3131, 655, \dots\}, T = 3968$

Given a 3CNF expression Φ if there exists an assignment A that satisfies Φ , prove that the set S and limit T as constructed in the previous slide, would have a solution:

Pick the items corresponding to the assignment. They will sum to T .

Conversely, if there is a subset that will sum to T , then pick the assignments corresponding to the items in the subset. They will satisfy Φ .

What about KNAPSACK?

Given $I = \{(w_1, v_1), \dots, (w_n, v_n)\}$, a limit W and a value V ,

Is there an $I' \subset I$ such that $\sum_{\{i \in I'\}} w_i \leq W$ and $\sum_{\{i \in I'\}} v_i \geq V$?

Wait...isn't it $O(nW)$? That sounds polynomial.

In NP?

Reducing SUBSET-SUM to KNAPSACK

31/32

Given $S = \{s_1, s_2, \dots, s_n\}$ and T .

Let $I = \{(s_1, s_1), (s_2, s_2), \dots, (s_n, s_n)\}$. Let $W = V = T$.

Boom. If you can solve KNAPSACK in polynomial time, you can solve SUBSET-SUM in polynomial time.

Questions?

Slides by Aaron Hillegass

Based on lectures by Abraham Ladha